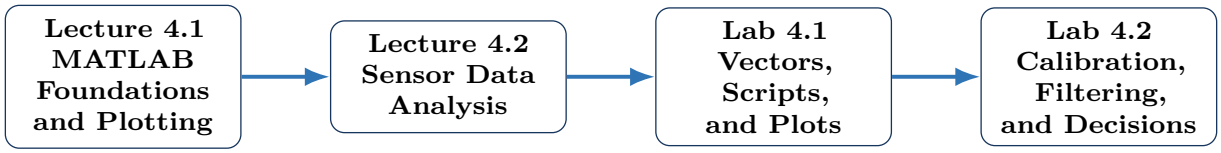
Week 4: MATLAB for Mechatronics

Lecture 4.1 and Lecture 4.2

ENGR 1105 | Fall 2026



Week 4 Goal

Use MATLAB to organize engineering data, perform calculations, create readable plots, calibrate sensor measurements, reduce noise, and identify operating states from real sensor data.

Meeting	Main focus
Lecture 4.1	MATLAB interface, variables, vectors, indexing, scripts, functions, plotting, and program flow
Lecture 4.2	Importing sensor data, validation, filtering, calibration, logical indexing, thresholds, and engineering communication
Lab 4.1	Build MATLAB scripts that calculate and plot engineering data
Lab 4.2	Analyze Week 3 sensor data and create a calibrated warning model

Opening Questions

1. Why is a plot often more useful than a long column of sensor values?
2. What is the difference between raw data and calibrated data?
3. How can MATLAB help us find a pattern that is difficult to see in the Serial Monitor?

Lecture 4.1

MATLAB Foundations and Engineering Plots

Monday, November 16, 2026

Learning Objectives

By the end of Lecture 4.1, you should be able to:

1. describe the roles of the Command Window, Workspace, Current Folder, and Editor;
2. create variables, vectors, and matrices;
3. use indexing and the colon operator;
4. distinguish matrix operations from element-by-element operations;
5. create and run a script;
6. use basic `if` and `for` structures; and
7. create an engineering plot with a title, labels, legend, and grid.

1. Why MATLAB?

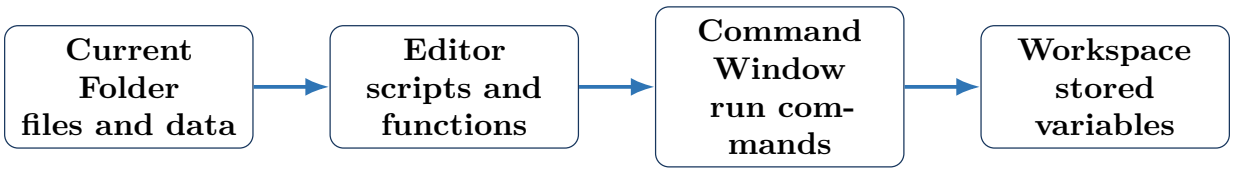
MATLAB is an engineering computing environment for numerical calculations, data analysis, visualization, modeling, and algorithm development. In mechatronics, it can help us:

- inspect Arduino sensor measurements;
- compare raw, filtered, and calibrated data;
- calculate engineering quantities;
- visualize motion, voltage, light, temperature, or distance;
- test decision rules before placing them in a controller.

MATLAB Is a Tool for Engineering Reasoning

MATLAB does not replace understanding. It makes repetitive calculations faster and helps us see patterns, but the engineer must still choose correct units, equations, assumptions, and interpretations.

2. The MATLAB Working Environment



MATLAB area	Purpose
Command Window	Enter and test individual commands.
Editor	Write, save, and run scripts or functions.
Workspace	View variables currently stored in memory.
Current Folder	View files available to the current MATLAB session.
Figures	Display plots and graphical results.

Current Folder Matters

MATLAB must be able to find the script or data file. Save related files in one project folder and make that folder the Current Folder.

3. Variables and Basic Calculations

MATLAB creates a variable when a value is assigned.

```
voltage = 5.0;
current = 0.012;
resistance = voltage / current;
```

The semicolon suppresses printed output. Without a semicolon, MATLAB displays the result in the Command Window.

```
distance = 0.75
distance = 0.75;
```

> **Assignment and Comparison**
>
> In MATLAB, `=` assigns a value. The operator `==` compares two values.

4. Useful Naming Rules

A variable name:

- begins with a letter;
- may contain letters, numbers, and underscores;
- cannot contain spaces;
- is case-sensitive.

Clear name	Less clear name
`distanceCm`	`d`
`sensorVoltage`	`x1`
`filteredData`	`temp`

5. Vectors

A vector stores multiple values in one variable.

```
distance = [20.8 20.1 21.0 19.7 20.4];
time = [0 0.2 0.4 0.6 0.8];
```

A row vector uses spaces or commas. A column vector uses semicolons.

```
rowVector = [1 2 3 4];
columnVector = [1; 2; 3; 4];
```

Why Vectors Matter

Most sensor experiments produce many measurements. Vectors allow one command to operate on an entire data set.

6. The Colon Operator

The colon operator creates evenly spaced values.

```
a = 1:5;           % [1 2 3 4 5]
b = 0:0.5:2;       % [0 0.5 1.0 1.5 2.0]
```

The general form is

`start : step : stop`.

The function `linspace()` creates a specified number of equally spaced points.

```
time = linspace(0,10,101);
```

7. Indexing

Indexing selects one or more elements.

```
distance = [20.8 20.1 21.0 19.7 20.4];

firstValue = distance(1);
thirdValue = distance(3);
middleValues = distance(2:4);
lastValue = distance(end);
```

> **MATLAB Starts at 1**
>
> The first MATLAB index is 1, not 0.

8. Matrices and Columns

A matrix stores values in rows and columns.

```
data = [
  0.0  20.8
  0.2  20.1
  0.4  21.0
  0.6  19.7
];

time = data(:,1);
distance = data(:,2);
```

The colon in `data(:,1)` means “all rows from column 1.”

9. Element-by-Element Operations

MATLAB distinguishes matrix operations from element-by-element operations.

Operation	Meaning	Example
`*`	matrix multiplication	`A*B`
`.*`	multiply corresponding elements	`voltage.*current`
`/`	matrix right division	advanced linear algebra use
`./`	divide corresponding elements	`distance./time`
`^`	matrix power	`A^2`
`.^`	raise each element to a power	`speed.^2`

Kinetic Energy for Several Speeds

For a mass $m = 0.50\,\text{kg}$ and speeds

$$v = [0,\ 1,\ 2,\ 3]\ \text{m/s},$$

calculate

$$KE = \frac{1}{2}mv^2.$$

```
m = 0.50;
v = [0 1 2 3];
KE = 0.5 * m * v.^2;
```

The result is

$$KE = [0,\ 0.25,\ 1.00,\ 2.25]\ \text{J}.$$

10. Useful Built-In Functions

```
minimumValue = min(distance);
maximumValue = max(distance);
averageValue = mean(distance);
numberOfValues = length(distance);
standardDeviation = std(distance);
```

These functions describe a data set without writing a loop.

11. Scripts

A script is a saved sequence of MATLAB commands stored in an `.m` file.

```
%% Week 4 distance example

time = [0 0.2 0.4 0.6 0.8];
distance = [20.8 20.1 21.0 19.7 20.4];

averageDistance = mean(distance);

plot(time,distance,"o-")
grid on
xlabel("Time (s)")
ylabel("Distance (cm)")
title("Ultrasonic Distance Measurements")
```

Why Use a Script?

A script creates a repeatable record of the analysis. The same steps can be rerun after the data changes.

12. Comments and Sections

Comments begin with a percent symbol.

```
% This line explains the next calculation
averageDistance = mean(distance);
```

A double percent symbol creates a code section.

```
%% Import Data
%% Filter Data
%% Create Plot
```

Sections help organize a long script and allow selected parts to run independently.

13. Clearing the Environment

Common commands include:

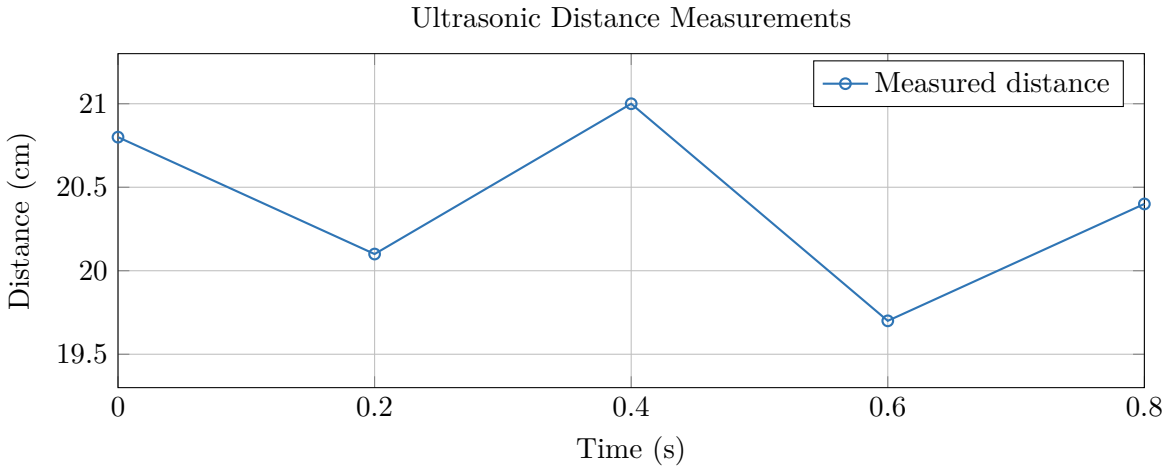
```
clear
clc
close all
```

- `clear` removes variables from the Workspace.
- `clc` clears displayed Command Window text.
- `close all` closes open figure windows.

14. Engineering Plots

A clear plot should include:

- the measured or calculated data;
- axis labels with units;
- a descriptive title;
- a legend when multiple data sets are shown;
- a grid when it helps interpretation.




```
plot(time,distance,"o-","LineWidth",1.5)
grid on
xlabel("Time (s)")
ylabel("Distance (cm)")
title("Ultrasonic Distance Measurements")
```

15. Multiple Data Sets

```
plot(time,rawDistance,"o-")
hold on
plot(time,filteredDistance,"LineWidth",1.5)
hold off

legend("Raw","Filtered","Location","best")
```

The Plot Should Communicate

Avoid titles such as “Graph 1.” State what is measured and include units on both axes.

16. Decision Statements

MATLAB uses `if`, `elseif`, and `else` for decisions.

```
distanceCm = 28;

if distanceCm < 15
    state = "danger";
elseif distanceCm < 35
    state = "warning";
else
    state = "safe";
end
```

Comparison operators include

`==`, `=`, `>`, `<`, `>=`, `<=`.

Logical operators include

`&`, `|`, `~`.

17. Loops

A `for` loop repeats a section of code.

```
for k = 1:length(distance)
    fprintf("Measurement %d: %.1f cm\n", ...
        k,distance(k))
end
```

Prefer Vector Operations When They Are Clear

MATLAB is designed to work with vectors and matrices. A loop is useful when each step depends on the previous step or when the repeated procedure is easier to understand as a loop.

18. Simple Functions

A function accepts inputs and returns outputs.

```
function voltage = adcToVoltage(adcValue)
    voltage = adcValue * 5.0 / 1023.0;
end
```

Usage:

```
v = adcToVoltage(614);
```

Functions help avoid copying the same calculation into many locations.

Lecture 4.1 Exit Questions

1. What is the difference between a script and a Command Window command?
2. What does `data(:,2)` select?
3. Why is `.^` used when squaring every value in a vector?
4. What four elements should appear on an engineering plot?

Lecture 4.2

Sensor Data Analysis and Reliable Decisions

Tuesday, November 17, 2026

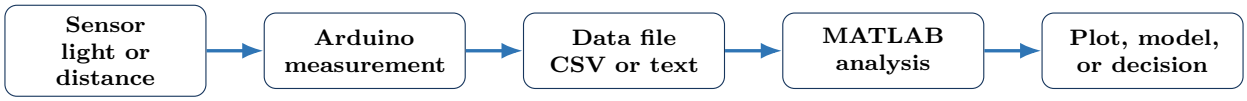
Learning Objectives

By the end of Lecture 4.2, you should be able to:

1. import numerical data from a file;
2. separate data columns using indexing;
3. identify invalid readings and outliers;
4. apply a moving average;
5. perform a simple linear calibration;
6. use logical indexing to classify operating states; and
7. create a plot that compares raw, filtered, calibrated, and reference data.

19. From Arduino to MATLAB

During Week 3, the Arduino displayed sensor readings in the Serial Monitor. MATLAB provides a larger workflow for storing and analyzing those values.



Keep Raw Data

Do not replace the original measurements with processed values. Store raw data and create separate variables for filtered or calibrated results.

20. Importing a CSV File

The Week 4 package contains

`sample_sensor_data.csv`.

Import the numerical values:

```
data = readmatrix("sample_sensor_data.csv");

time = data(2:end,1);
rawDistance = data(2:end,2);
referenceDistance = data(2:end,3);
```

The first row contains column headings, so the numerical data begin at row 2.

Alternative Import Method

MATLAB's Import Tool can preview a file and generate import code. For a repeatable engineering workflow, save the generated commands in a script.

21. Inspect the Data First

Before filtering or calibrating, inspect the data.

```
size(data)
min(rawDistance)
max(rawDistance)
mean(rawDistance)
plot(time,rawDistance,"o-")
grid on
```

Questions to ask:

- Are the units correct?
- Are the data columns in the expected order?
- Are any values missing?
- Are any values outside the sensor's useful range?
- Does the plot show sudden suspicious jumps?

22. Valid and Invalid Measurements

For an ultrasonic sensor with a useful range of approximately 2 cm to 400 cm:

```
valid = rawDistance >= 2 & rawDistance <= 400;

validTime = time(valid);
validDistance = rawDistance(valid);
```

The expression creates a logical vector containing `true` and `false` values.

Logical Indexing

Suppose

$$d = [10,\ -1,\ 30,\ 500,\ 25].$$

Then

`valid = d >= 2 & d <= 400`

produces

[`true`, `false`, `true`, `false`, `true`].

Using `d(valid)` returns

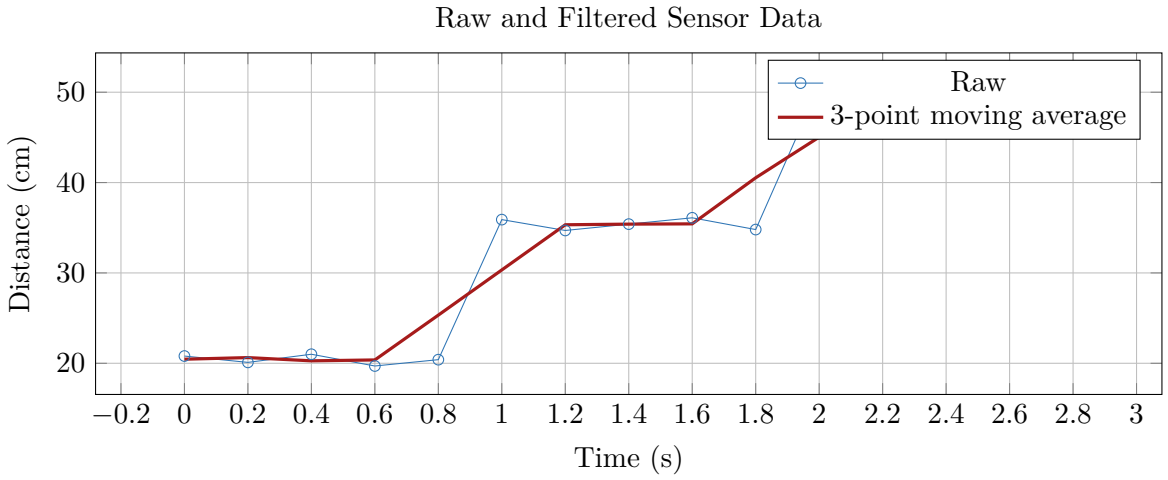
$$[10,\ 30,\ 25].$$

23. Moving Average Filtering

A moving average replaces each value with the average of nearby values.

```
filteredDistance = movmean(rawDistance,3);
```

The value 3 is the window length.



Filtering Tradeoff

A larger window reduces random variation but can blur sudden real changes. The filter should match the speed of the physical system.

24. Mean and Standard Deviation

```
averageValue = mean(rawDistance);
spread = std(rawDistance);
```

The mean describes the center of the data. Standard deviation describes the spread of repeated measurements.

25. Calibration with a Linear Model

Suppose the sensor has a consistent offset or scale error. A linear calibration model is

$$y = mx + b,$$

where:

- x is the raw sensor value;
- y is the calibrated estimate;
- m is the scale factor;
- b is the offset.

MATLAB can estimate m and b using `polyfit()`.

```
p = polyfit(rawDistance,referenceDistance,1);

scale = p(1);
offset = p(2);

calibratedDistance = polyval(p,rawDistance);
```

Reading the Calibration Coefficients

If MATLAB returns

$$p = [0.98,\ 0.55],$$

the calibration model is

$$d_{\text{calibrated}} = 0.98 d_{\text{raw}} + 0.55.$$

26. Calibration Plot

```
plot(rawDistance,referenceDistance,"o")
hold on

xFit = linspace(min(rawDistance), ...
    max(rawDistance),100);
yFit = polyval(p,xFit);

plot(xFit,yFit,"LineWidth",1.5)
hold off
grid on

xlabel("Raw sensor distance (cm)")
ylabel("Reference distance (cm)")
legend("Calibration data","Linear fit", ...
    "Location","best")
```

A calibration plot should show whether a straight line is a reasonable model.

27. Error

Measurement error can be calculated as

$$e = y_{\text{measured}} - y_{\text{reference}}.$$

```
rawError = rawDistance - referenceDistance;
calibratedError = calibratedDistance ...
    - referenceDistance;

meanAbsoluteRawError = mean(abs(rawError));
meanAbsoluteCalibratedError = ...
    mean(abs(calibratedError));
```

Evaluate the Model

A calibration is useful when the calibrated error is smaller than the raw error for the data conditions that matter.

28. Classifying Sensor States

Logical expressions can classify every measurement at once.

```
danger = calibratedDistance < 25;

warning = calibratedDistance >= 25 ...
    & calibratedDistance < 40;

safe = calibratedDistance >= 40;
```

Count the number of values in each state:

```
dangerCount = sum(danger);
warningCount = sum(warning);
safeCount = sum(safe);
```

A logical value `true` behaves like 1 when summed.

29. Create a State Label

```
state = strings(size(calibratedDistance));

state(danger) = "danger";
state(warning) = "warning";
state(safe) = "safe";
```

30. Create a Results Table

```
results = table( ...
    time, ...
    rawDistance, ...
    filteredDistance, ...
    calibratedDistance, ...
    state);

disp(results)
```

Tables connect variable names with columns and make results easier to inspect.

31. Complete Analysis Script

```
%% Week 4 sensor-data analysis

clear
clc
close all

data = readmatrix("sample_sensor_data.csv");

time = data(2:end,1);
rawDistance = data(2:end,2);
referenceDistance = data(2:end,3);

valid = rawDistance >= 2 ...
    & rawDistance <= 400;

time = time(valid);
rawDistance = rawDistance(valid);
referenceDistance = ...
    referenceDistance(valid);

filteredDistance = movmean(rawDistance,3);

p = polyfit(rawDistance, ...
    referenceDistance,1);

calibratedDistance = ...
    polyval(p,rawDistance);

danger = calibratedDistance < 25;
warning = calibratedDistance >= 25 ...
    & calibratedDistance < 40;
safe = calibratedDistance >= 40;
```

```
state = strings(size(calibratedDistance));
state(danger) = "danger";
state(warning) = "warning";
state(safe) = "safe";

figure
plot(time,rawDistance,"o-")
hold on
plot(time,filteredDistance, ...
    "LineWidth",1.5)
plot(time,referenceDistance,"--", ...
    "LineWidth",1.5)
hold off

grid on
xlabel("Time (s)")
ylabel("Distance (cm)")
title("Week 4 Sensor Data Analysis")
legend("Raw","Moving average", ...
    "Reference","Location","best")

results = table(time,rawDistance, ...
    filteredDistance, ...
    calibratedDistance,state);

disp(results)
```

32. Engineering Communication

A strong analysis should state:

1. what was measured;
2. the units;
3. how invalid data were handled;
4. which filtering or calibration method was used;
5. the thresholds used for decisions;
6. what the plot shows;
7. the limitations of the result.

Interpret the Result

A filtered plot looks smoother than the raw plot. Does that automatically mean the filtered result is more accurate? Explain.

33. Common MATLAB Problems

Observed problem	First checks
MATLAB cannot find the file	Check Current Folder, file name, spelling, and extension.
Undefined variable	Run the earlier script section that creates the variable.
Matrix dimensions do not agree	Check vector sizes and whether element-by-element operators are needed.
Plot is empty	Check that variables contain numerical data and have matching lengths.
Index exceeds array bounds	Check the size of the vector or matrix and remember indexing begins at 1.
Calibration result is unreasonable	Check units, reference data, column order, and whether a linear model is appropriate.

34. Week 4 Summary

Core Ideas

- MATLAB stores engineering data in variables, vectors, matrices, and tables.
- Indexing selects individual values, ranges, rows, or columns.
- Element-by-element operators apply calculations to corresponding vector elements.
- Scripts create a saved and repeatable engineering analysis.
- A useful plot has labels, units, title, legend, and readable formatting.
- Logical indexing selects valid data and classifies operating states.
- A moving average reduces random variation but can slow or blur the response.
- Calibration converts raw measurements into estimates that better match reference values.
- Raw data, processed data, assumptions, and limitations should remain clearly separated.

Lecture 4.2 Exit Questions

1. What does `data(2:end,3)` select?
2. Why should raw and filtered data be stored in different variables?
3. What does `polyfit(x,y,1)` calculate?
4. What is the tradeoff when the moving-average window becomes larger?
5. How can logical indexing separate danger, warning, and safe measurements?

Included Week 4 Files

The package contains `sample_sensor_data.csv` and `week4_sensor_demo.m`. Keep these files in the same folder as the Week 4 MATLAB script while completing examples.